

Python Programming

Class Test I & II Solutions (Even Sem 2024-2025)

Class Test - I (Date: 25/3/2025)

Q1 (a) Describe Membership and Identity operators in python with appropriate example.

[7 Marks]

1. Identity Operators

Identity operators compare the memory locations of two objects to check if they are exactly the same object, not just if they have the same value.

Operator	Description	Example
<code>is</code>	Returns True if both variables point to the same object.	<code>x is y</code>
<code>is not</code>	Returns True if both variables point to different objects.	<code>x is not y</code>

2. Membership Operators

Membership operators are used to test if a sequence (string, list, tuple, set, dictionary) contains a specific value or item.

Operator	Description	Example
<code>in</code>	Returns True if a specified value is present in the object.	<code>x in y</code>

`not in`

Returns True if a specified value is NOT present in the object.

`x not in y`

```
# --- Identity Example ---
a = [1, 2, 3]
b = [1, 2, 3]
c = a

print(a is c)      # Same object in memory
print(a is b)      # Different objects, even though values match
print(a == b)      # Values match

# --- Membership Example ---
fruits = ["apple", "banana"]
print("apple" in fruits)
print("mango" not in fruits)
```

► TERMINAL OUTPUT

```
True
False
True
True
True
```

Q1 (b) List features of Python.

[3 Marks]

- **Simple & Easy to Learn:** Clean, English-like syntax.
- **Interpreted:** Code is executed line-by-line, aiding in easy debugging.
- **Dynamically Typed:** Variables don't require explicit data type declarations.
- **Platform Independent:** Write once, run anywhere (Windows, Mac, Linux).
- **Extensive Standard Library:** "Batteries included" with massive built-in modules.

Q1 (c) [OR] WAP to print the Fibonacci sequence.**[3 Marks]**

```
# Python program to print Fibonacci sequence up to n terms
n = 7 # Assume user input is 7

a, b = 0, 1
print("Fibonacci Sequence:")
for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b # Update values
```

► TERMINAL OUTPUT

```
Fibonacci Sequence:
0 1 1 2 3 5 8
```

Q2 (a) Explain different loops available in python with suitable examples.**[7 Marks]**

Python provides two primary looping constructs to execute a block of statements repeatedly.

1. The for Loop

Used for iterating over a sequence (such as a list, tuple, dictionary, set, string) or a range of numbers. It automatically handles the iteration and stops when the sequence is exhausted.

```
# Example: for loop using range
for i in range(1, 4):
    print("Iteration:", i)
```

► TERMINAL OUTPUT

```
Iteration: 1
Iteration: 2
Iteration: 3
```

2. The while Loop

Executes a set of statements continuously as long as a specified boolean condition is True. The condition must eventually become False, or it will result in an infinite loop.

```
# Example: while loop
count = 1
while count <= 3:
    print("Count is:", count)
    count += 1 # Increment counter to avoid infinite loop
```

► TERMINAL OUTPUT

```
Count is: 1
Count is: 2
Count is: 3
```

Q2 (b) WAP using eval() to create a calculator.

[3 Marks]

```
# Simple Calculator using eval()
expression = "5 * 4 + 2" # Simulating input: input("Enter a mathematical expression: ")

try:
    result = eval(expression)
    print("Expression:", expression)
    print("Result:", result)
except Exception as e:
    print("Invalid Expression")
```

► TERMINAL OUTPUT

```
Expression: 5 * 4 + 2  
Result: 22
```

Q2 (c) [OR] WAP to print all prime number between a specified interval.**[3 Marks]**

```
lower = 10 # Simulating user input  
upper = 20 # Simulating user input  
  
print(f"Prime numbers between {lower} and {upper}:")  
for num in range(lower, upper + 1):  
    if num > 1: # Primes are greater than 1  
        for i in range(2, int(num**0.5) + 1):  
            if (num % i) == 0:  
                break # Divisible, not prime  
        else:  
            print(num, end=" ")
```

► TERMINAL OUTPUT

```
Prime numbers between 10 and 20:  
11 13 17 19
```

Class Test - II (Date: 22/4/2025)

Q1 (a) What are Lists? Explain append(), extend(), pop() methods of list with suitable examples of each.

[7 Marks]

Lists are ordered, mutable (changeable) collections of items enclosed in square brackets `[]`. They can hold items of heterogeneous data types.

Method	Description	Example
append(item)	Adds a single element to the very end of the list. It increases the length of the list by exactly one.	<pre>L = [1, 2] L.append(3) # L becomes [1, 2, 3]</pre>
extend(iterable)	Adds all elements of an iterable (like another list, tuple, or string) to the end of the current list.	<pre>L = [1, 2] L.extend([3, 4]) # L becomes [1, 2, 3, 4]</pre>
pop(index)	Removes and returns the element at the specified index. If no index is provided, it removes and returns the <i>last</i> item.	<pre>L = [10, 20, 30] L.pop(1) # Returns 20, L becomes [10, 30]</pre>

Q1 (b) Write a python program to count uppercase and lowercase In given string "Python is FUN "

[3 Marks]

```
text = "Python is FUN "
upper_count = 0
lower_count = 0

for char in text:
    if char.isupper():
        upper_count += 1
    elif char.islower():
        lower_count += 1

print("Uppercase count:", upper_count)
print("Lowercase count:", lower_count)
```

► TERMINAL OUTPUT

```
Uppercase count: 5  
Lowercase count: 7
```

Q1 (c) [OR] What is String Slicing? Find the output of below program.

[3 Marks]

String Slicing is a technique to extract a specific substring from a string by specifying a start index, stop index, and step size using the syntax `string[start:stop:step]`.

Output Tracing:

```
# a)  
name = "PYTHONISFUN"  
print("a)", name[2:5])  
# Explanation: Index 2 is 'T', 3 is 'H', 4 is 'O' (5 is exclusive).  
  
# b)  
my_list = [1, 2, 3, 4, 5, 6, 7, 8]  
print("b)", my_list[-6:-1:2])  
# Explanation: -6 index is '3'. -1 index is '8' (exclusive).  
# Range values: 3, 4, 5, 6, 7. Step is 2.
```

► TERMINAL OUTPUT

```
a) THO  
b) [3, 5, 7]
```

Q2 (a) Define Tuples. Demonstrate How to change tuple values? [7 Marks]

Definition

A **Tuple** is a sequence data type in Python, similar to a list, but it is **immutable**. Once a tuple is created, its elements cannot be added, removed, or modified. They are enclosed in parentheses `()`.

How to change tuple values

Because tuples are immutable, you cannot directly assign a new value to an index (e.g., `t[0] = 5` will raise an error). To alter a tuple, you must perform a **workaround**:

1. Convert the tuple into a list.
2. Modify the desired values in the list.
3. Convert the list back into a tuple.

```
# Original Tuple
my_tuple = ("Apple", "Banana", "Cherry")
print("Original:", my_tuple)

# 1. Convert to List
temp_list = list(my_tuple)

# 2. Modify the List
temp_list[1] = "Blueberry" # Changing 'Banana' to 'Blueberry'
temp_list.append("Orange") # Adding a new item

# 3. Convert back to Tuple
my_tuple = tuple(temp_list)

print("Modified:", my_tuple)
```

► TERMINAL OUTPUT

```
Original: ('Apple', 'Banana', 'Cherry')
Modified: ('Apple', 'Blueberry', 'Cherry', 'Orange')
```

Q2 (b) WAP for implementing the programming language details using set operations.

[3 Marks]


```
# Defining sets of programming languages known by two people
dev1 = {"Python", "Java", "C++"}
dev2 = {"Java", "JavaScript", "Python"}

# Union (All languages combined)
print("All languages:", dev1 | dev2)

# Intersection (Common languages)
print("Common languages:", dev1 & dev2)

# Difference (Languages Dev1 knows but Dev2 doesn't)
print("Only Dev1 knows:", dev1 - dev2)
```

► TERMINAL OUTPUT

```
All languages: {'Java', 'C++', 'Python', 'JavaScript'}
Common languages: {'Java', 'Python'}
Only Dev1 knows: {'C++'}
```

Q2 (c) [OR] WAP to create a set of automobile component by using set methods add 2 items in same set then remove any 1 item.

[3 Marks]

```
# Create initial set
auto_parts = {"Engine", "Steering"}
print("Initial Set:", auto_parts)

# Add 2 items
auto_parts.add("Brakes")
auto_parts.add("Gearbox")
print("After adding:", auto_parts)

# Remove 1 item
auto_parts.remove("Steering") # discard() can also be used
print("After removing:", auto_parts)
```

► TERMINAL OUTPUT

```
Initial Set: {'Steering', 'Engine'}
After adding: {'Gearbox', 'Steering', 'Brakes', 'Engine'}
```

```
After removing: {'Gearbox', 'Brakes', 'Engine'}
```